

第 14 章 | TensorFlow 核心機制與客製化實作

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線

本章從 Keras 高階 API 的使用經驗出發，帶你理解 TensorFlow 底層機制，並練習自訂損失函數、特殊神經層與可部署的資料流程。

本章聚焦於「拆解底層機制」：不只學會呼叫 API，也要理解背後的數學邏輯與架構權衡。掌握這些機制後，你就能面對客製化需求，做出有依據的設計決策。

本章學習目標：

掌握引擎演進：釐清 TensorFlow v1 靜態圖（Blueprint 模式）與 v2 動態圖（Eager Execution）的本質差異。

加速效能最佳化：理解 `@tf.function` 如何透過 AutoGraph 將 Python 邏輯轉化為 C++ 高效靜態圖。

深究自動微分：實作 `tf.GradientTape` 核心邏輯，理解其「反向累積 (Reverse Accumulation)」之運算優勢。

靈活建模選型：熟練運用 Functional API 與 Subclassing 建立非線性架構，並學會處理訓練與推論的邏輯分支。

可部署的資料管線：掌握 Feature Columns（包含 Embedding 機制）與 Estimator API 的自動存檔與部署流程。

2. 重要術語中英對照表

繁體中文	英文原文
計算圖	Computation Graph
即時執行 / 動態圖	Eager Execution
圖編譯機制	AutoGraph
有向無環圖	Directed Acyclic Graph (DAG)
自動微分	Automatic Differentiation
反向累積	Reverse Accumulation
權重初始化	Weight Initialization
模型容量	Model Capacity
特徵行	Feature Columns
嵌入行	Embedding Columns
檢查點	Checkpointing

3. 第一模組：TensorFlow 的大腦——計算圖 (Computation Graph)

計算圖的實務意義

TensorFlow 的核心運作依賴於「有向無環圖 (DAG)」。將數學公式轉換為節點（運算）與連線（張量流動）的目的，是為了讓電腦具備「反向追蹤」能力，這是自動微分的基礎。

核心觀念說明：發包企劃書 vs. 即時執行

TF v1 靜態圖（發包企劃書）：如同在動工前畫好完整的建築藍圖，必須透過 `Session` 才能執行，除錯極為困難。

TF v2 動態圖（Eager Execution）：現行主流模式，寫一行、跑一行，運作邏輯與一般 Python/NumPy 無異，開發直覺且易於除錯。

以公式 $z = 2 \times (a - b) + c$ 為例：在圖中， a 與 b 會先匯入減法節點（ $r1$ ），結果再進入乘法節點（ $r2$ ），最後與 c 匯合於加法節點得出 z 。

圖編譯加速：`@tf.function`

雖然動態圖便於除錯，但運算速度慢。透過在函式加上 `@tf.function` 裝飾器，TensorFlow 會啟動 AutoGraph 機制，將 Python 邏輯編譯成高效的靜態 C++ 圖。

學習提醒：初學者最常在 `@tf.function` 內部宣告 `tf.Variable`，這會導致重複初始化報錯（`ValueError`）。變數必須在函式外宣告，函式內僅負責運算。

實際意義

效能與除錯的權衡：研發初期應使用動態圖除錯，當模型穩定後，再透過 `@tf.function` 封裝核心運算邏輯以獲得生產級的效能提升。

底層透明化：理解「圖」的存在，能幫助工程師診斷計算路徑中的效能瓶頸。

4. 第二模組：權重初始化與自動微分錄音機

權重載體與初始化策略

權重必須儲存於 `tf.Variable`。初始化方式決定了訓練的成敗。

Xavier (Glorot) 初始化：工業界標準。它透過輸入 (n_{in}) 與輸出 (n_{out}) 神經元數量，動態平衡梯度的變異數（例如：`tf.keras.initializers.GlorotUniform()`）。

打破對稱性：若權重全設為 0，模型將無法學習。正確的初始化能防止梯度消失或爆炸。

自動微分：`tf.GradientTape`

`tf.GradientTape` 採用 反向累積 (Reverse Accumulation) 機制，這比前向累積在處理「多參數、單標籤 (Loss)」的神經網路時效率更高。

錄音機比喻：

錄製：`with tf.GradientTape() as tape:` 記錄前向傳播軌跡。

倒帶：`tape.gradient(loss, weights)` 使用連鎖律回傳梯度。

更新：`optimizer.apply_gradients` 將梯度套用於變數。

實務警告：錄音帶預設呼叫一次即銷毀。若需計算多個標籤的梯度，必須宣告 `persistent=True`。

實際意義

自定義訓練環節：當 `model.fit()` 無法滿足特殊的強化學習或對抗生成網路 (GAN) 邏輯時，`GradientTape` 是唯一且必須掌握的底層操作工具。

計算效率：理解反向累積機制能讓你明白為何神經網路訓練可以規模化。

5. 第三模組：模型容量與非線性架構實戰

決策邊界與模型容量

「模型容量」決定了模型逼近複雜函數的能力。

XOR 問題：單層線性模型無法切分交錯資料。

非線性轉換：透過隱藏層與 ReLU 等激勵函數「扭曲空間」，模型才能畫出彎曲的決策邊界。

建模 API 的選型

Functional API（接水管）：透過 `tf.keras.Input` 定義源頭，可輕鬆實現「多輸入/多輸出」或「殘差跳接」。業界 80% 的複雜問題皆以此解決。

Model Subclassing（高度自訂）：繼承 `tf.keras.Model` 並定義 `call()`。

專業提示：必須使用 `call(self, inputs, training=False)` 簽名。例如在開發自定義層時，利用 `training` 旗標確保「隨機雜訊」或「Dropout」僅在訓練階段觸發，而非推論階段。

實際意義

架構決策：架構師應根據團隊協作需求選型。Functional API 提供最佳的序列化與視覺化支援；Subclassing 則為科研人員開發新穎演算法（如 `NoisyLinear`）的首選。

6. 第四模組：可部署的特徵管線與部署 (Feature Columns)

當處理 Auto MPG 這類混合型結構化資料時，手動預處理（如 `pandas`）會導致部署時資料邏輯難以同步。

Feature Columns 技術流程

`numeric_column`：連續數值。

`bucketized_column`：將連續值分桶（如年份區間化）。

`indicator_column`：低維度類別的 One-hot 編碼。

`embedding_column`（關鍵）：當類別數量龐大（高基數特徵）時，將稀疏向量映射為低維稠密浮點向量，顯著降低計算負擔並提升模型泛化力。

Estimator API：生產線利器

Checkpointing 機制：自動存檔，支援斷點續傳，防止伺服器斷電導致心血泡湯。

邏輯封裝：特徵轉換邏輯與模型一同打包，確保生產環境與開發環境資料處理 100% 一致。

實際意義

部署穩定性：採用 Feature Columns 與 Estimator 能將資料預處理邏輯「模型化」，減少了生產環境對外部清理腳本的依賴，這是邁向生產級 AI 的最後一哩路。

7. 實戰導引：Colab 操作步驟與錯誤排除

關鍵操作步驟

環境預熱：`!pip install mlxtend` 以繪製決策區域。

定義 `input_fn`：將 `pandas/tf.data` 轉化為字典格式，對接 Feature Columns。

轉換模型：練習使用 `tf.keras.estimator.model_to_estimator` 將開發好的模型賦予工業部署能力。

常見實作錯誤檢查表

- `[] ValueError: tf.function ... tried to create variables`

解決： 將 `tf.Variable` 宣告移出被 `@tf.function` 裝飾的函式。

- `[] RuntimeError: Graph is finalized`

原因： TF2 在 Colab 中的偶發 Bug。_解決：_ 「重新啟動執行階段 (Restart Runtime)」。

- `[] Gradient is None`

解決： 檢查張量是否為 `tf.Variable`。若是常數，請手動呼叫 `tape.watch(tensor)`。

- `[] Persistent Tape Memory Leak`

提醒： 若使用 `persistent=True`，必須在結尾顯式刪除 `del tape` 以釋放記憶體。

8. 動手練習與自我檢查表

練習題

手動梯度實作：修改計算圖，使用 `GradientTape` 計算 $z=(a+b) \times c$ 對 a 的偏導數。

自定義 `NoisyLinear` 層：實作公式 $w(x+\epsilon)+b$ ，其中 ϵ 是標準差為 0.1 的高斯雜訊。請確保雜訊僅在 `training=True` 時添加，並觀察其對 XOR 問題決策邊界的平滑效果。

混合輸入模型：使用 Functional API 建立一個模型，同時接收「汽車數值特徵」與「產地 Embedding 向量」。

自我檢查表

- `[]` 我能解釋為什麼自動微分要用「反向累積」而非前向累積嗎？
- `[]` 我是否理解 `embedding_column` 在處理類別資料時優於 One-hot 的場景？
- `[]` 我是否清楚 `@tf.function` 內部禁止的變數操作邏輯？
- `[]` 我能否在 `call()` 方法中根據 `training` 狀態區分不同的運算邏輯？
- `[]` 我理解 `Estimator` 相比於 `model.fit` 在工業大規模部署時的實務優勢了嗎？